Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра ИИТ

Лабораторная работа №1
По дисциплине: «КиСБ»
Тема: «Исследование алгоритмов генерации псевдослучайных последовательностей и методов тестирования чисел на простоту»

Выполнил:
Студент 4-го курса
Группы АС-65
Макарский А.Э.
Проверил:
Хацкевич А.С.

Брест 2026

Цель работы: освоить программную реализацию и практическое применение алгоритмов генерации псевдослучайных последовательностей (РСП), методов их статистической оценки, а также вероятностных и детерминированных тестов простоты чисел с построением доказуемо простых чисел.
Вариант 10
1. Генерация и тестирование псевдослучайных последовательностей (файл "Лабораторная работа №1 (ч.1)")
В рамках данного этапа необходимо разработать программный модуль генерации последовательностей и провести их первичный анализ.

1.1. Проверка примитивности многочленов

Для варианта 10 заданы три многочлена степени 5 над полем GF(2):

1. f1(x)=x5+x2+1$f1(x)=x5+x2+1$
2. f2(x)=x5+x4+x3+x2+1$f2(x)=x5+x4+x3+x2+1$
3. f3(x)=x5+1$f3(x)=x5+1$

poly_utils.py:

```
def poly_mod(a, b):

    while a.bit_length() >= b.bit_length():
        a ^= b << (a.bit_length() - b.bit_length())
    return a

def poly_mul(a, b):

    res = 0
    while b:
        if b & 1:
            res ^= a
        a <<= 1
        b >>= 1
    return res

def poly_mul_mod(a, b, mod):
    return poly_mod(poly_mul(a, b), mod)

def poly_pow_mod(base, exp, mod):
    """base^exp mod в GF(2)[x]."""
    res = 1
    while exp:
        if exp & 1:
            res = poly_mul_mod(res, base, mod)
        base = poly_mul_mod(base, base, mod)
        exp >>= 1
    return res

def factor_int(n):

    fs = []
    d = 2
    while d * d <= n:
        if n % d == 0:
            fs.append(d)
            while n % d == 0:
                n //= d
        d += 1
    if n > 1:
        fs.append(n)
    return fs

def is_irreducible(poly, degree):
```

```
    # 1. x^(2^deg) = x mod poly
    x = 0b10
    if poly_pow_mod(x, 2**degree, poly) != x:
        return False
    # 2. Для каждого простого p | degree: gcd(x^(2^(deg/p)) - x, poly) = 1
    for p in factor_int(degree):
        sub = poly_pow_mod(x, 2**(degree // p), poly) ^ x
        # gcd многочленов — алгоритм Евклида
        a, b = poly, sub
        while b != 0:
            a, b = b, poly_mod(a, b)
        if a.bit_length() > 1:  # gcd != 1
            return False
    return True

def is_primitive(poly, degree):

    if not is_irreducible(poly, degree):
        return False
    order = 2**degree - 1
    x = 0b10
    if poly_pow_mod(x, order, poly) != 1:
        return False
    for p in factor_int(order):
        if poly_pow_mod(x, order // p, poly) == 1:
            return False
    return True
```

```
Многочлен                          | неприводим | примитивен
-------------------------------------------------------------
f1: x^5 + x^2 + 1                  | True       | True
f2: x^5 + x^4 + x^3 + x^2 + 1      | True       | True
f3: x^5 + 1                        | False      | False
```

1.2. Синтез SG-генератора и подбор параметров

Для генерации битов используется комбинирование методом псевдослучайного прореживания (Shrinking Generator, SG). Реализован класс LFSR (линейный регистр сдвига) и класс ShrinkingGenerator.

```
Тестируем LFSR на основе f1 (x^5 + x^2 + 1)
seed = 00001 (bin) | Период = 31
seed = 00101 (bin) | Период = 31
seed = 10101 (bin) | Период = 31

Тестируем LFSR на основе f2 (x^5 + x^4 + x^3 + x^2 + 1)
seed = 00001 (bin) | Период = 31
```

1.3. Статистическое тестирование

Для проверки равномерности распределения использовались частотный тест (Monobit test) и критерий Хи-квадрат. Оценивалась доля единиц на выборках разной длины.

Длина (N)	Единиц	Нулей	Доля (p)	z-стат	Равном.?	Хи-квадрат
100	54	46	0.5400	0.800	True	0.640
500	254	246	0.5080	0.358	True	0.128
1000	502	498	0.5020	0.126	True	0.016
1984	1024	960	0.5161	1.437	True	2.065
5000	2569	2431	0.5138	1.952	True	3.809
10000	5161	4839	0.5161	3.220	False	10.368

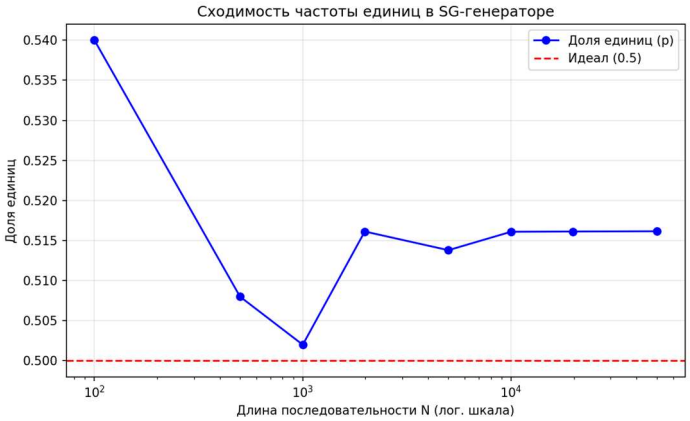
```
Примечания к таблице:
- 'Равном.?' = True, если |z| < 1.96 (доверительная вероятность 95%)
- 'Хи-квадрат' должен быть < 3.841 для alpha=0.05 (df=1)
```

1.4. Графики свойств последовательности

По результатам работы программы были построены графики

1. **График сходимости доли единиц к 0.5 в зависимости от длины последовательности.**
2. **График расположения единиц в первых 100 битах, показывающий отсутствие явных периодических структур.**





2. Тестирование на простоту и генерация больших простых чисел (файл "Лабораторная работа №1 (ч.2)")
На втором этапе реализован метод генерации доказуемо простых чисел Михалеску с параметром e=1/2*e*=1/2.
Суть метода: Число строится по формуле n=2tF+1*n*=2*tF*+1, где F*F* — произведение известных малых простых чисел, занимающее примерно m/2*m*/2 бит. Благодаря тому, что мы полностью знаем факторизацию n−1*n*−1, простота числа n*n* не просто предполагается, а математически доказывается с помощью критерия Поклингтона (Pocklington primality test)

mihailescu.py:

```
import random
import time
from math import gcd
from primality import SMALL_PRIMES, trial_division, miller_rabin, factor_small


def build_F(e_bits, max_attempts=200):
    """Строит число F с известной факторизацией (произведение малых простых).
    Размер F примерно e_bits бит."""
    for _ in range(max_attempts):
        F = 1
        factors = {}
        # Набираем произведение случайных малых простых
        while F.bit_length() < e_bits:
            p = random.choice(SMALL_PRIMES[:40])
            F *= p
            factors[p] = factors.get(p, 0) + 1
        # Проверяем, что не перелетели сильно за диапазон
        if F.bit_length() <= e_bits + 1:
            return F, factors
    raise RuntimeError("Не удалось построить F")


def pocklington_prove(n, known_factors):
    """Доказательство простоты n по Поклингтону.
    n-1 = 2 * t * F. known_factors — все известные простые делители n-1."""
    for q in known_factors:
        found = False
        # Ищем "свидетеля" (a)
        for _ in range(100):
            a = random.randrange(2, n - 1)
            if pow(a, n - 1, n) != 1:
                continue
            if gcd(pow(a, (n - 1) // q, n) - 1, n) == 1:
                found = True
                break
```

```
        if not found:
            return False  # не удалось доказать для этого q
    return True


def generate_prime_mihailescu(m_bits, e=0.5, s=200):
    """Генерация доказуемо простого числа по методу Михалеску.

    Параметры:
      m_bits — требуемая битовая длина числа n
      e      — параметр (для варианта 10: 0.5)
      s      — сколько кандидатов перебрать за раз
    """
    t0 = time.time()

    # ШАГ 2: F из диапазона [2^(e*m), 2^(e*m+1)]
    e_bits = max(8, int(e * m_bits))
    F, F_factors = build_F(e_bits)

    # ШАГ 3: t ∈ (2^(m-1)/(2F), 2^m/(2F))
    lo = (1 << (m_bits - 1)) // (2 * F) + 1
    hi = (1 << m_bits) // (2 * F)

    if hi <= lo:
        # F получилось слишком большим, рекурсивно пробуем еще раз
        return generate_prime_mihailescu(m_bits, e, s)

    attempts = 0
    # ШАГ 4: перебор кандидатов
    for _ in range(s):
        attempts += 1
        t = random.randint(lo, max(lo, hi - 1))
        n = 2 * t * F + 1  # Формула Михалеску

        # 4.1: Пробные деления (быстрый отсев)
        if not trial_division(n):
            continue

        # 4.2: Фильтр Рабина-Миллера (если он не прошел, дальше смысла нет)
        if not miller_rabin(n, k=5):
            continue

        # 4.3: Доказательство простоты (Поклингтон)
        # n-1 = 2 * t * F. Нам нужны все простые делители (t и F)
        known = set(F_factors.keys())
        t_f = factor_small(t)
        known |= set(t_f.keys())

        if pocklington_prove(n, known):
            return {
                'n': n, 'F': F, 't': t, 'attempts': attempts,
                'time': time.time() - t0, 'proven': True,
            }

    return {
        'n': None, 'F': F, 't': None, 'attempts': attempts,
        'time': time.time() - t0, 'proven': False,
    }
```

Разряд (бит)	Время (с)	F (бит)	Кандидатов	Доказано	sympy.isprime
32	0.0000	17	12	True	True
64	0.0000	33	23	True	True
128	0.0000	65	1	True	True
256	0.0156	128	51	True	True
512	0.0625	257	92	True	True

Вывод

В ходе выполнения лабораторной работы были изучены принципы построения генераторов псевдослучайных последовательностей на основе регистров сдвига (LFSR) и метода прореживания (SG-генератор). Была исследована проблема вырождения генератора при использовании регистров с одинаковыми периодами и найден способ достижения максимального периода. Также был освоен метод Михалеску, позволяющий генерировать числа, простоту которых можно строго доказать, что критически важно для криптографических задач.